

Lab Assignment-7.1

2303A54012
A.Sanjay B47A

Task Description #1 (Syntax Errors — Missing Parentheses in Print Statement) Task:

Provide a Python snippet with a missing parenthesis in a print statement (e.g., `print "Hello"`). Use AI to detect and fix the syntax error.

Requirements:

Run the given code to observe the error.

Apply AI suggestions to correct the syntax.

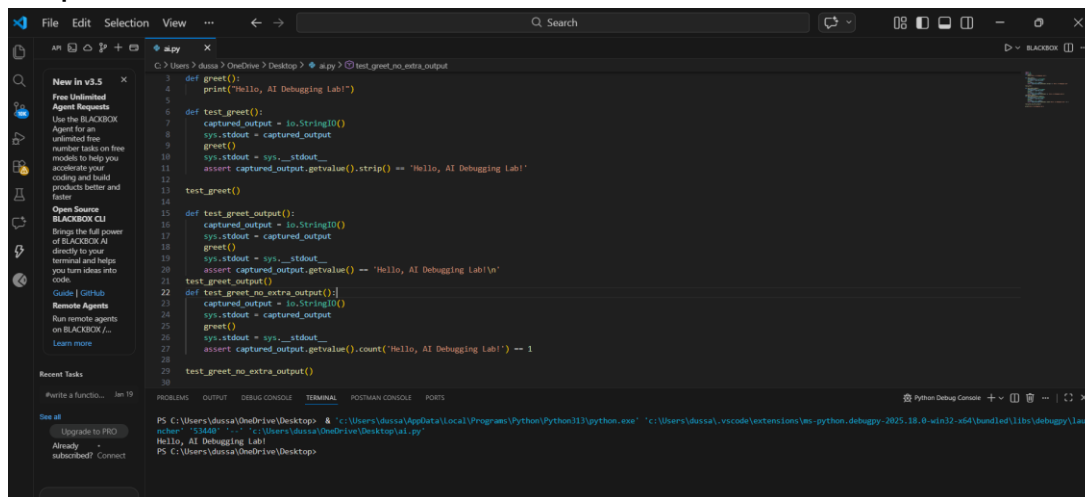
Expected Output #1 :

- Corrected code with proper syntax and AI explanation.

Prompt:

#Detect and fix the syntax error in this Python code:

#def greet(): print 'Hello, AI Debugging Lab!' ; greet() #Explain the issue, provide corrected Python 3 code, and include 3 assert tests to verify correct output.



```
1 def greet():
2     print("Hello, AI Debugging Lab!")
3
4 def test_greet():
5     captured_output = io.StringIO()
6     sys.stdout = captured_output
7     greet()
8     sys.stdout = sys._stdout__
9     assert captured_output.getvalue().strip() == 'Hello, AI Debugging Lab!'
10
11 test_greet()
12
13 def test_greet_output():
14     captured_output = io.StringIO()
15     sys.stdout = captured_output
16     greet()
17     sys.stdout = sys._stdout__
18     assert captured_output.getvalue() == 'Hello, AI Debugging Lab!\n'
19
20 test_greet_output()
21
22 def test_greet_no_extra_output():
23     captured_output = io.StringIO()
24     sys.stdout = captured_output
25     greet()
26     sys.stdout = sys._stdout__
27     assert captured_output.getvalue().count('Hello, AI Debugging Lab!') == 1
28
29 test_greet_no_extra_output()
30
```

Explanation:

The issue in the original code is that it uses Python 2 syntax for the print statement. In Python 3, print is a function and requires parentheses.

Task Description #2 (Incorrect condition in an If Statement)

Task:

Supply a function where an if-condition mistakenly uses = instead of == to identify and fix the issue.

Bug: Using assignment (=) instead of comparison (==)

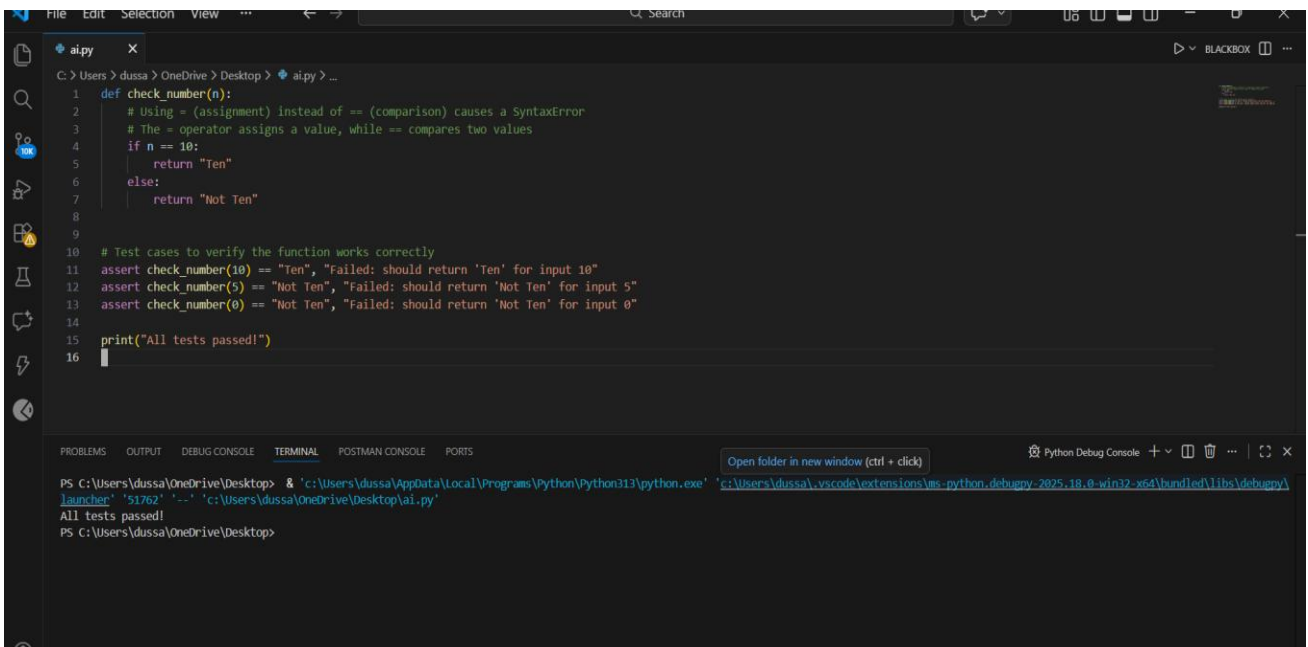
```
def check_number(n):  
    if n = 10:  
        return "Ten"  
    else:
```

S.

Prompt used:

Detect and fix the error in this Python code: def check_number(n): if n = 10: return "Ten" else: return "Not Ten". #Explain why using = causes a bug, provide corrected code using '==', and include 3 assert tests to verify the function works.

Code:



```
1 def check_number(n):  
2     # Using = (assignment) instead of == (comparison) causes a SyntaxError  
3     # The = operator assigns a value, while == compares two values  
4     if n = 10:  
5         return "Ten"  
6     else:  
7         return "Not Ten"  
8  
9  
10 # Test cases to verify the function works correctly  
11 assert check_number(10) == "Ten", "Failed: should return 'Ten' for input 10"  
12 assert check_number(5) == "Not Ten", "Failed: should return 'Not Ten' for input 5"  
13 assert check_number(0) == "Not Ten", "Failed: should return 'Not Ten' for input 0"  
14  
15 print("All tests passed!")  
16
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL POSTMAN CONSOLE PORTS

PS C:\Users\dussa\OneDrive\Desktop> & 'c:\Users\dussa\AppData\Local\Programs\Python\Python313\python.exe' 'c:\Users\dussa\.vscode\extensions\ms-python.debugpy-2025.18.0-win32-x64\bundled\libs\debugpy\launcher' '51762' '--' 'c:\Users\dussa\OneDrive\Desktop\ai.py'
All tests passed!
PS C:\Users\dussa\OneDrive\Desktop>

Explanation :

The error occurs because `=` is an assignment operator, not a comparison operator. In an if condition, Python requires `==` to compare values, and using `=` causes a `SyntaxError`. Replacing `=` with `==` correctly checks whether `n` is equal to 10, allowing the function to work properly.

Task Description #3 (Runtime Error — File Not Found)

Task:

Provide code that attempts to open a non-existent file and crashes. Use AI to apply safe error handling.

Bug: Program crashes if file is missing

```
def read_file(filename):  
    with open(filename, 'r') as f:  
        return f.read()  
print(read_file("nonexistent.txt"))
```

Requirements:

- Implement a try-except block suggested by AI.
- Add a user-friendly error message.
- Test with at least 3 scenarios: file exists, file missing, invalid Path.

Expected Output #3:

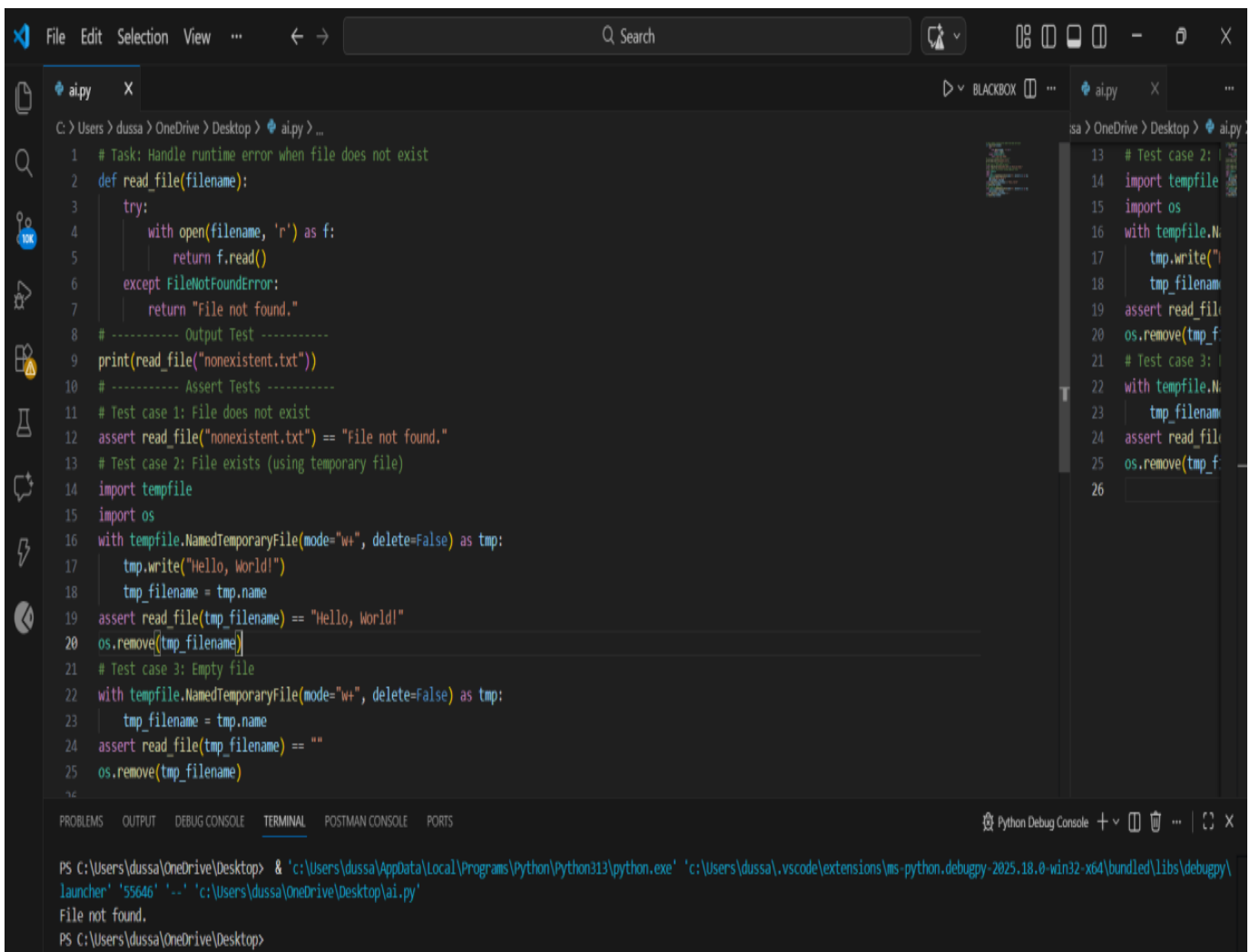
- Safe file handling with exception management.

Prompt:

#Analyze this Python code that crashes when a file does not exist: `def read_file(filename):
 with open(filename, 'r') as f: return f.read()`

#Fix the runtime error using try—except, add a user-friendly error message, and include 3 assert tests for file exists, file missing, and invalid path cases.

code:



The image shows a Visual Studio Code editor window with a Python file named `ai.py` open. The file contains a function `read_file(filename)` that attempts to open a file and return its contents. It uses a `try-except` block to catch `FileNotFoundError` and return a user-friendly message "File not found." instead of crashing. The script includes several test cases: a file that does not exist, a file created with `tempfile` and `os` modules, and an empty file. The terminal at the bottom shows the command to run the script, which successfully executes and prints "File not found."

```
1 # Task: Handle runtime error when file does not exist
2 def read_file(filename):
3     try:
4         with open(filename, 'r') as f:
5             return f.read()
6     except FileNotFoundError:
7         return "File not found."
8
9 # ----- Output Test -----
10 print(read_file("nonexistent.txt"))
11 # ----- Assert Tests -----
12 # Test case 1: File does not exist
13 assert read_file("nonexistent.txt") == "file not found."
14 # Test case 2: File exists (using temporary file)
15 import tempfile
16 import os
17 with tempfile.NamedTemporaryFile(mode="w+", delete=False) as tmp:
18     tmp.write("Hello, World!")
19     tmp_filename = tmp.name
20 assert read_file(tmp_filename) == "Hello, World!"
21 os.remove(tmp_filename)
22 # Test case 3: Empty file
23 with tempfile.NamedTemporaryFile(mode="w+", delete=False) as tmp:
24     tmp_filename = tmp.name
25 assert read_file(tmp_filename) == ""
26 os.remove(tmp_filename)
```

Terminal output:

```
PS C:\Users\dussa\OneDrive\Desktop> & 'c:\Users\dussa\AppData\Local\Programs\Python\Python313\python.exe' 'c:\Users\dussa\.vscode\extensions\ms-python.debugpy-2025.18.0-win32-x64\bundled\libs\debugpy\launcher' '55646' '-.' 'c:\Users\dussa\OneDrive\Desktop\ai.py'
File not found.
PS C:\Users\dussa\OneDrive\Desktop>
```

Explanation:

The program crashes because it tries to open a file that does not exist, which raises a `FileNotFoundError`. By using a `try—except` block, the exception is caught and handled safely instead of stopping the program. This allows the function to return a userfriendly error message while continuing execution.

TASK:

Give a class where a non-existent method is called (e.g., `obj.undefined_method()`). Use AI to debug and fix.

Bug: Calling an undefined

method class Car:

```
def start(self):
```

```
    return "Car started"
```

```
my_car = Car()
```

```
print(my_car.drive()) #
```

`drive()` is not defined

Requirements:

- Students must analyze whether to define the missing method or correct the method call.
- Use 3 assert tests to confirm the corrected class works.

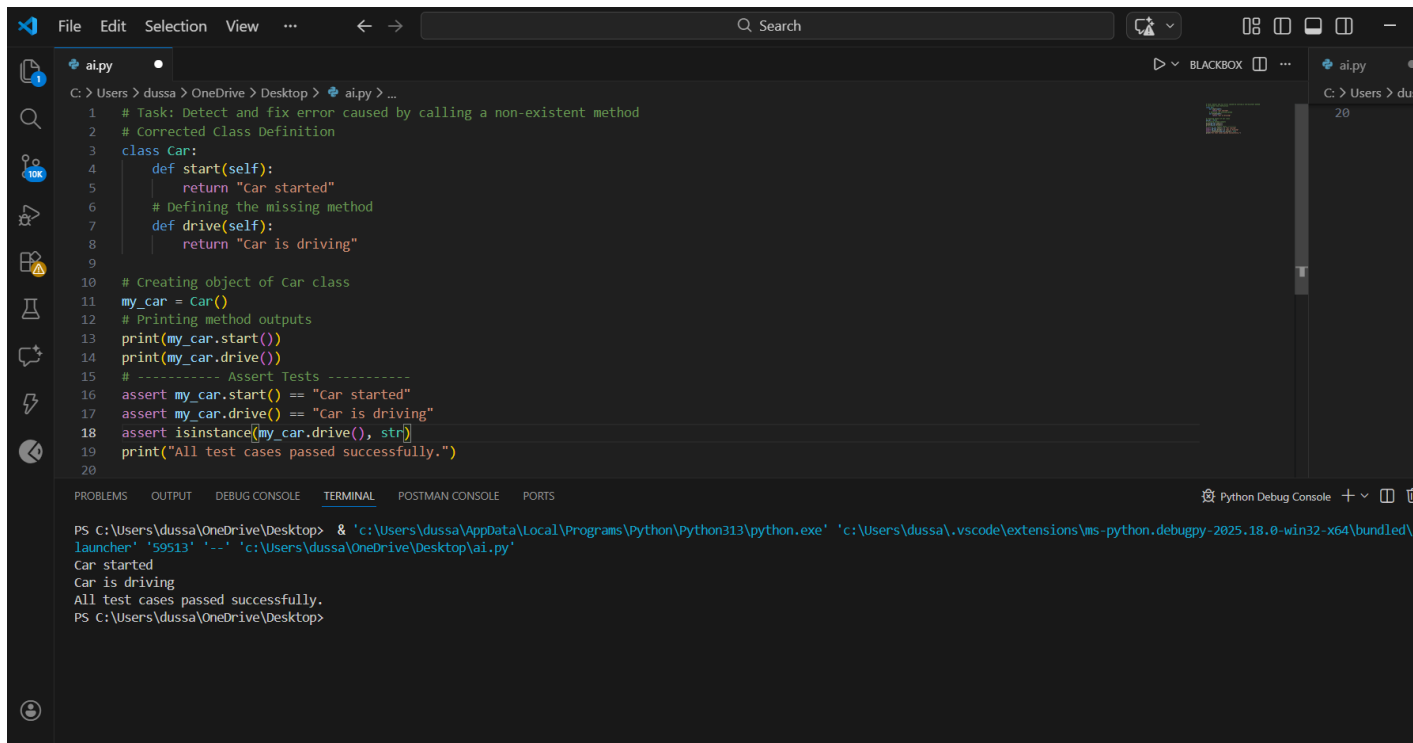
Expected Output #4:

- Corrected class with clear AI explanation.

Prompt used:

#Detect and fix the error in this Python code where a nonexistent method is called: `class Car: def start(self): return "Car started" my_car = Car(); print(my_car.drive())` #Explain the cause of the error, decide whether to define the missing method or correct the method call, provide corrected code, and include 3 assert tests to verify the class works properly.

Code :



The screenshot shows a VS Code editor with a file named `ai.py` open. The file contains a Python class `Car` with two methods: `start` and `drive`. The `start` method returns the string "Car started", and the `drive` method returns the string "Car is driving". Below the class definition, an object `my_car` is created, and both methods are called. The output of these calls is printed, and then two assertions are made to verify the outputs. Finally, a print statement indicates that all tests passed successfully.

```
1 # Task: Detect and fix error caused by calling a non-existent method
2 # Corrected Class Definition
3 class Car:
4     def start(self):
5         return "Car started"
6     # Defining the missing method
7     def drive(self):
8         return "Car is driving"
9
10 # Creating object of Car class
11 my_car = Car()
12 # Printing method outputs
13 print(my_car.start())
14 print(my_car.drive())
15 # ----- Assert Tests -----
16 assert my_car.start() == "Car started"
17 assert my_car.drive() == "Car is driving"
18 assert isinstance(my_car.drive(), str)
19 print("All test cases passed successfully.")
20
```

The terminal output at the bottom shows the execution of the script, confirming that the methods are called correctly and the assertions pass.

```
PS C:\Users\dussa\OneDrive\Desktop> & 'c:\Users\dussa\AppData\Local\Programs\Python\Python313\python.exe' 'c:\Users\dussa\.vscode\extensions\ms-python.debugpy-2025.18.0-win32-x64\bundle
launcher' '59513' '--' 'c:\Users\dussa\OneDrive\Desktop\ai.py'
Car started
Car is driving
All test cases passed successfully.
PS C:\Users\dussa\OneDrive\Desktop>
```

Explanation:

The error occurs because the method `drive()` is called on the `Car` object, but it is not defined in the `Car` class, which raises an

Task Description #5 (TypeError — Mixing Strings and Integers in Addition)

Task:

Provide code that adds an integer and string `5" + 2)` causing a `TypeError`. Use AI to resolve the bug.

Bug: `TypeError` due to mixing string

and integer `def add_five(value): return`

`value + 5 print(add_five("`

`10"))`

Requirements:

- Ask AI for two solutions: type casting and string concatenation.

- Validate with 3 assert test cases.

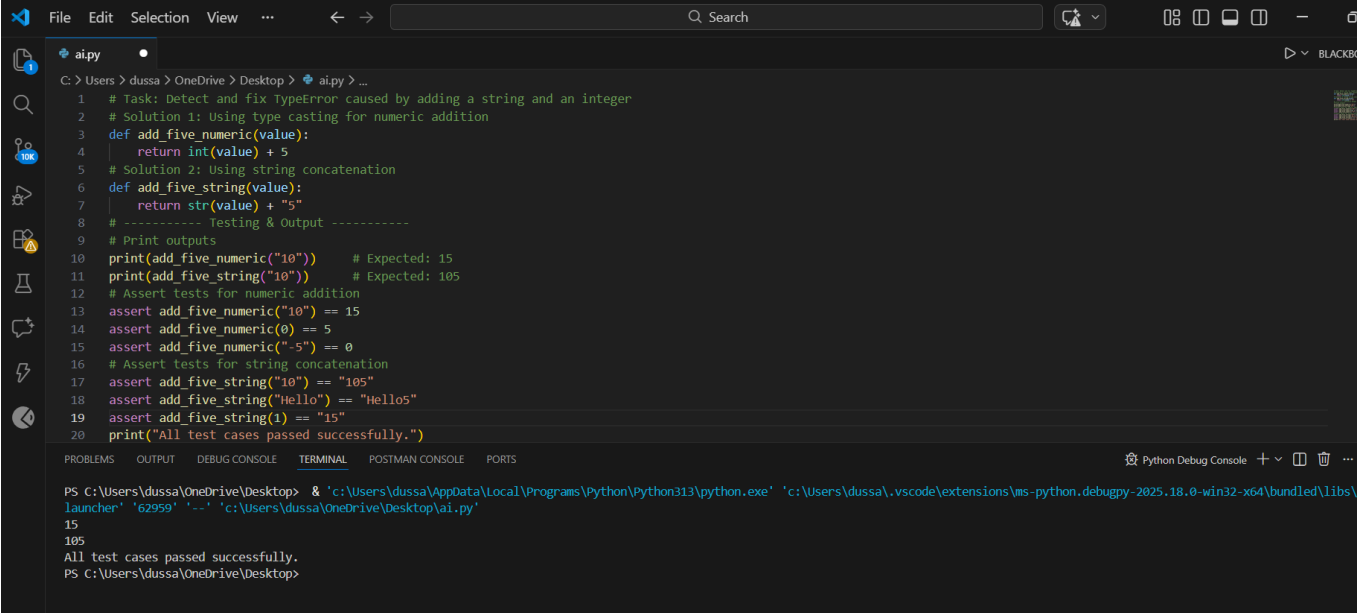
Expected Output #5:

- Corrected code that runs successfully for multiple inputs.

Prompt used:

#Detect and fix the TypeError in this Python code caused by adding a string and an integer: def add_five(value): return value + 5 ; print(add_five("10"))#Explain why the error occurs, provide two solutions (one using type casting for numeric addition and one using string concatenation), and include 3 assert tests to verify both solutions work correctly.

Code:



```
1 # Task: Detect and fix TypeError caused by adding a string and an integer
2 # Solution 1: Using type casting for numeric addition
3 def add_five_numeric(value):
4     return int(value) + 5
5 # Solution 2: Using string concatenation
6 def add_five_string(value):
7     return str(value) + "5"
8 # ----- Testing & Output -----
9 # Print outputs
10 print(add_five_numeric("10")) # Expected: 15
11 print(add_five_string("10")) # Expected: 105
12 # Assert tests for numeric addition
13 assert add_five_numeric("10") == 15
14 assert add_five_numeric(0) == 5
15 assert add_five_numeric("-5") == 0
16 # Assert tests for string concatenation
17 assert add_five_string("10") == "105"
18 assert add_five_string("Hello") == "Hello5"
19 assert add_five_string(1) == "15"
20 print("All test cases passed successfully.")
```

PS C:\Users\dussa\OneDrive\Desktop> & 'c:\Users\dussa\AppData\Local\Programs\Python\Python313\python.exe' 'c:\Users\dussa\.vscode\extensions\ms-python.debugpy-2025.18.0-win32-x64\bundle\libs\launcher' '62959' '--' 'c:\Users\dussa\OneDrive\Desktop\ai.py'

15
105
All test cases passed successfully.
PS C:\Users\dussa\OneDrive\Desktop>

Explanation:

The error occurs because Python cannot add a string and an integer, which raises a TypeError. One solution is to convert the input to an integer using type casting so numeric addition can be performed. Another solution is to convert the value to a string and use string concatenation instead, depending on the intended behavior.